

Insertion Sort (Line-by-Line Explanation with Edge Cases)

Code with Explanations and Comments

```
import java.io.*;
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in); // To read input from the user
        int t = s.nextInt(); // Number of test cases

        while(t-- > 0){ // Run for each test case
            int n = s.nextInt(); // Read array size
            int a[] = new int[n]; // Declare array

            for(int i = 0; i < n; i++){ // Read array elements
                a[i] = s.nextInt();
            }

            // Insertion Sort Logic Starts
            for(int i = 1; i < n; i++){
                int k = a[i]; // Current element to be inserted
                int j = i - 1; // Start from previous index

                // Shift elements to right until correct spot is found
                while(j >= 0 && a[j] > k){
                    a[j + 1] = a[j];
                    j--;
                }

                // Insert element at correct position
                a[j + 1] = k;

                // Print the final inserted index position
                System.out.print((j + 1) + (i == n - 1 ? "" : " "));
            }
            System.out.println(); // Newline after each test case
        }
    }
}
```

DETAILED EXPLANATION

1. The first loop runs through test cases.
2. For each array, we apply insertion sort.
3. For every element, we check where it fits in the sorted left portion.

4. We shift larger elements rightward and insert the key in its correct place.
5. Instead of printing the sorted array, we print the final position ($j + 1$) where each element was inserted.

EXAMPLE

Input:

1

5

4 3 2 1 5

Output:

0 0 0 4

Explanation:

- Element 3 inserted at index 0
- Element 2 inserted at index 0
- Element 1 inserted at index 0
- Element 5 stays at index 4

EDGE CASES

1■■■ Empty array ($n = 0$)

Input: []

Output: (no output, handles gracefully)

2■■■ Single element array

Input: [5]

Output: (no output, since loop starts at $i = 1$)

3■■■ Already sorted array

Input: [1, 2, 3, 4]

Output: 1 2 3 (each element stays where it is)

4■■■ Reverse sorted array

Input: [5, 4, 3, 2, 1]

Output: 0 0 0 0 (each new element goes to start)

5■■■ Array with duplicate elements

Input: [3, 3, 3, 3]

Output: 1 2 3 (since equal elements are not moved)

6■■■ Large input (Stress Test)

Input: 1000 elements

Output: Still $O(n^2)$ time — works but slower

TIME & SPACE COMPLEXITY

-
- Best Case: $O(n)$ — when array already sorted
 - Worst Case: $O(n^2)$ — when array is reverse sorted
 - Space Complexity: $O(1)$ — in-place sorting
-

CONCLUSION

- Insertion sort is simple and effective for small arrays or partially sorted data.
- Here, instead of outputting sorted array, we print the positions where insertions happened.
- Stable and easy to understand algorithm.